Submitted to the Annals of Probability

COMPOSITIONAL CERTIFICATES FOR REVERSIBLE NEURAL NETWORKS

BY ANONYMOUS AUTHOR,

Roop generates inverse programs over fixed-width machine integers. We give a Lean certificate calculus for networks assembled from additive coupling layers. A layer preserves one tensor and adds an arbitrary function of it into the other. The update function remains opaque. For every tensor width, element width, and finite list of layers, Lean proves that forward execution followed by reverse execution restores the initial state, and conversely. The theorem removes whole-network arithmetic expansion from the reversibility argument. It does not yet connect generated Weave code or emitted CUDA kernels to the certificate calculus.

1. Machine arithmetic. Let $B_n = \mathrm{BitVec}(n)$. Addition and subtraction in B_n are modulo 2^n. Hence, for every $a, b \in B_n$,

$$(a + b) - b = a, \qquad (a - b) + b = a. \tag{1}$$

Let $T_{d,n} = \mathrm{Fin}(d) \to B_n$. These identities extend pointwise to $T_{d,n}$. The Lean proof uses function extensionality and does not enumerate the d coordinates.

2. Coupling layers. For $f : T_{d,n} \to T_{d,n}$, define

$$U_f(q, p) = (q, p + f(q)), \qquad V_f(q, p) = (q + f(p), p). \tag{2}$$

Their inverses replace addition by subtraction. No condition on the internal computation of f is needed. In particular, f need not be linear, injective, differentiable, or reversible. The preserved component supplies the argument used to recompute f during reverse execution.

The Lean development represents each layer as ‘CouplingLayer‘. Its method ‘certificate‘ chooses U_f or V_f. The implementation resides in RoopLean’s certificate library.

3. Arbitrary finite networks. For $L = [\ell_1, \ldots, \ell_k]$, ‘couplingNetwork L‘ constructs a ‘Program‘ from the individual certificates. Its generic certificate yields

$$\mathrm{undo}_L(\mathrm{exec}_L(s)) = s, \qquad \mathrm{exec}_L(\mathrm{undo}_L(s)) = s. \tag{3}$$

The theorem quantifies over d, n, k, all layer directions, and all update functions. Its proof checks the program structure and invokes the certificate of each layer. It does not normalize an Einsum, matrix product, activation, or tensor element inside f.

A Rust regression checks the Lean module. The theorem is therefore checked in the repository.

4. Proof boundary. The theorem covers additive coupling programs over modular bitvectors. It does not establish numerical accuracy, gradient correctness, alias safety in generated source, resource bounds, or a correspondence between the Roop compiler and this representation. The next theorem must show that a generated Weave layer preserves one tensor and performs the pointwise modular update named by its ‘CouplingLayer‘ certificate. After that correspondence is established, increasing network depth adds checked transformations rather than expanded arithmetic proofs.

MSC2020 subject classifications. Primary 68N20.

Keywords and phrases. reversible computation, formal verification, neural networks.